1) Scenario: parsing Table conflict in LL(1)

In LL(1) parser, the parsing table is constructed using the FIRST & FOLLOW sets of the grammar. the first set determines that can appear at the begging of strings derived from a production, while the FOLLOW set contains terminals a that can appear immediately after a non-terminal. During parsing table construction each production is placed based on these sets.

A conflict occurs when more than one production is entered into the same table cell for a non-terminal and an input symbol. As a result, the parser cannot uniquely determine which of the production should be applied.

Since LL(1) parsing is deterministic, every table must contain only one production.

Decision:

The grammar should be modified by eliminating left recursion, performing left factoring, & ensuring that the FIRST & FOLLOW sets do not overlap.

2) scenario:

Incorrect parse Tree Generation

The syntax analyzer constructs a parse tree by applying grammar productions according to the input tokens for the expression id+id-id, the parse tree determines the order which operations are verified.

If the grammar does not specify operator associativity, the expression may be grouped incorrectly, the expression may be grouped incorrectly. For example it may be interpreted as id + (id - id) instead of the correct (id + id) - id. This results in the incorrect evaluation because the operators are not associated properly.

Grammar rules directly influence the parse tree. A grammar does not enforce associativity valid parse tree for the same expression.

Decision:

The grammar should enforce left associativity for addition & subtraction for example:

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow id$

3) Scenario:

Invalid expression handling

The lexical Analyzer correctly identifies the token in the expression a+b as the identifiers and operators. However, the syntax analyzer checks whether these token follow the grammar rules.

The error occurs because the grammar expects an operand after the '+' operator. But the another operator '*' appears instead. Therefore the token sequence violates the grammar making it impossible to construct a valid parse tree.

In predictive parsing (LL), the parser detects the error when no valid production exists in the parsing table for the current input symbol. Without proper recovery, the paper stops after reporting the error.

Decision:

The parser should implement error detection & the recovery techniques, such as panic mode recovery, parse level recovery. using the FOLLOW sets.

4) Ambiguity in the conditional statements.

The grammar

$S \rightarrow$ if ϵ then s

$s \rightarrow$ if ϵ then s else

$S \rightarrow a$

is ambiguous because the parser cannot determine which if the statement an else belongs to. This is known as dangling else problem.

When constructing the parsing table, conflicts because both productions begin with the same prefix: if ϵ then s.

The parser cannot decide whether to reduce using the first production or continue parsing the second producting connecting Else.

Decision:

The grammar should be rewritten to the. Distinguish matched & unmatched statements, for example.

→ S → m/u

→ m → if e then m else u/a

→ u → if e then s/ if c then u else u.

5) operator precedence conflict.

The grammar

E → E+E

E → E*E

E → id

is ambiguous because it does not define operator precedence or associativity

for the input id + id * id, two different parse trees are possible;

(id + id) * id

id + (id * id)

Operator precedence and associativity are essential to ensure the expressions are evaluated correctly.

Decision:-

The grammar should be rewritten to enforce precedence.

For example:

E → E+T | T

T → T * F | F

F → id

Here:

* Multiplication has the higher prcdence then the addition

* Addition & Multiplication are left associate.